An Analysis of Graph neural network

Graph neural network

$$a^{(l)} = \sigma!\left(W^{(l)} a^{(l-1)} + b^{(l)}\right)$$

Graph neural network -- Part 1

Scalability and distributed training Training GNNs on large graphs presents computational challenges that differ substantially from those in other neural network architectures. Two broad problem settings arise: training on a single very large graph (such as a social network with billions of nodes), and training on collections of large individual graphs (such as molecular systems requiring higher-order interaction modeling). For the single large graph setting, sampling-based methods reduce memory requirements by operating on subgraphs rather than the full graph. Cluster-GCN partitions the graph into subgraphs using graph partitioning algorithms, training the model on each partition in sequence. GraphSAINT instead samples subgraphs stochastically during training, constructing unbiased estimators for the full graph loss. Distributed frameworks such as DistDGL extend training to multi-machine settings, managing the inter-machine communication required for neighborhood aggregation when a graph's nodes are distributed across machines. For tasks such as atomic simulations, a different bottleneck arises: GNN architectures that model higher-order interactions between triplets or quadruplets of atoms are memory-intensive at the level of individual graphs, making standard data parallelism — where each GPU processes a separate sample — insufficient when individual graphs exceed single-device memory. Graph Parallelism addresses this by distributing a single input graph across multiple GPUs, partitioning nodes and edges across devices rather than distributing samples across devices.. This approach enabled GNNs with hundreds of millions to billions of parameters to be trained on atomic systems that would otherwise exceed single-device memory, and has been applied to the development of large-scale universal interatomic potentials.

Graph neural network -- Part 2

α u v = exp ■ (LeakyReLU (a T [W x u ■ W x v ■ e u v])) ∑ z ∈ N u exp ■ (LeakyReLU (a T [W x u ■ W x z ■ e u z])) {\displaystyle \alpha _{uv}={\frac {\exp({\text{LeakyReLU}}\left(\mathbf {a} ^{T}[\mathbf {W} \mathbf {x} _{u}\Vert \mathbf {W} \mathbf {x} _{v}\Vert \mathbf {e} _{uv}]\right))}{\sum _{z\in N_{u}}\exp({\text{LeakyReLU}}\left(\mathbf {a} ^{T}[\mathbf {W} \mathbf {x} _{u}\Vert \mathbf {W} \mathbf {x} _{z}\Vert \mathbf {e} _{uz}]\right))}}}

Graph neural network -- Part 3

Local pooling layers Local pooling layers coarsen the graph via downsampling. Subsequently, several learnable local pooling strategies that have been proposed are presented. For each case, the input is the initial graph represented by a matrix X {\displaystyle \mathbf {X} } of node features, and the graph adjacency matrix A {\displaystyle \mathbf {A} } . The output is the new matrix X ′ {\displaystyle \mathbf {X} '} of node features, and the new graph adjacency matrix A ′ {\displaystyle \mathbf {A} '} .

Graph neural network -- Part 4

where a {\displaystyle \mathbf {a} } is a vector of learnable weights, · T {\displaystyle \cdot ^{T}} indicates transposition, e u v {\displaystyle \mathbf {e} _{uv}} are the edge features (if present), and LeakyReLU {\displaystyle {\text{LeakyReLU}}} is a modified ReLU activation function. Attention coefficients are then normalized via softmax to make them easily comparable across different nodes. A GCN can be seen as a special case of a GAT where attention coefficients are not learnable, but fixed and equal to the edge weights w u v {\displaystyle w_{uv}} .

Graph neural network -- Part 5

where H {\displaystyle \mathbf {H} } is the matrix of node representations h u {\displaystyle \mathbf {h} _{u}} , X {\displaystyle \mathbf {X} } is the matrix of node features x u {\displaystyle \mathbf {x} _{u}} , σ (·) {\displaystyle \sigma (\cdot)} is an activation function (e.g., ReLU), A ~ {\displaystyle {\tilde {\mathbf {A} }}} is the graph adjacency matrix with the addition of self-loops, D ~ {\displaystyle {\tilde {\mathbf {D} }}} is the graph degree matrix with the addition of self-loops, and Θ {\displaystyle \mathbf {\Theta } } is a matrix of trainable parameters. In particular, let A {\displaystyle \mathbf {A} } be the graph adjacency matrix: then, one can define A ~ = A + I {\displaystyle {\tilde {\mathbf {A} }}=\mathbf {A} +\mathbf {I} } and D ~ i i = ∑ j ∈ V A ~ i j {\displaystyle {\tilde {\mathbf {D} }}_{ii}=\sum _{j\in V}{\tilde {A}}_{ij}} , where I {\displaystyle \mathbf {I} } denotes the identity matrix. This normalization ensures that the eigenvalues of D ~ − 1 2 A ~ D ~ − 1 2 {\displaystyle {\tilde {\mathbf {D} }}^{-{\frac {1}{2}}}{\tilde {\mathbf {A} }}{\tilde {\mathbf {D} }}^{-{\frac {1}{2}}}} are bounded in the range [0 , 1] {\displaystyle [0,1]} , avoiding numerical instabilities and exploding/vanishing gradients. A limitation of GCNs is that they do not allow multidimensional edge features e u v {\displaystyle \mathbf {e} _{uv}} . It is however possible to associate scalar weights w u v {\displaystyle w_{uv}} to each edge by imposing A u v = w u v {\displaystyle A_{uv}=w_{uv}} , i.e., by setting

each nonzero entry in the adjacency matrix equal to the weight of the corresponding edge.

Graph neural network -- Part 6

Message passing layers are permutation-equivariant layers mapping a graph into an updated representation of the same graph. Formally, they can be expressed as message passing neural networks (MPNNs). Let G = (V , E) {\displaystyle G=(V,E)} be a graph, where V {\displaystyle V} is the node set and E {\displaystyle E} is the edge set. Let N u {\displaystyle N_{u}} be the neighbourhood of some node u ∈ V {\displaystyle u\in V} . Additionally, let x u {\displaystyle \mathbf {x} _{u}} be the features of node u ∈ V {\displaystyle u\in V} , and e u v {\displaystyle \mathbf {e} _{uv}} be the features of edge (u , v) ∈ E {\displaystyle (u,v)\in E} . An MPNN layer can be expressed as follows:

Graph neural network -- Part 7

h u (l + 1) = GRU (m u (l + 1) , h u (l)) {\displaystyle \mathbf {h} _{u}^{(l+1)}={\text{GRU}}(\mathbf {m} _{u}^{(l+1)},\mathbf {h} _{u}^{(l)})}

Graph neural network -- Part 8

Graph-based representation of text helps to capture deeper semantic relationships between words. Many studies have used graph networks to enhance performance in various text processing tasks such as text classification, question answering, Neural Machine Translation (NMT), event extraction, fact verification, etc.

Graph neural network -- Part 9

When viewed as a graph, a network of computers can be analyzed with GNNs for anomaly detection. Anomalies within provenance graphs often correlate to malicious activity within the network. GNNs have been used to identify these anomalies on individual nodes and within paths to detect malicious processes, or on the edge level to detect lateral movement.